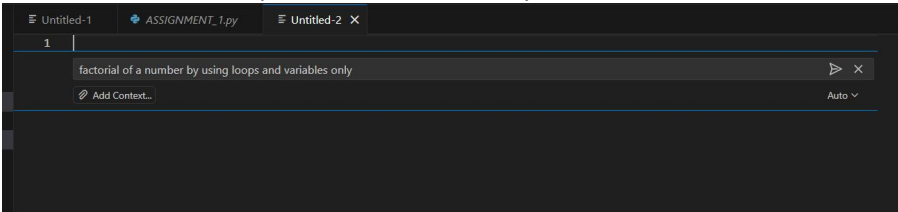
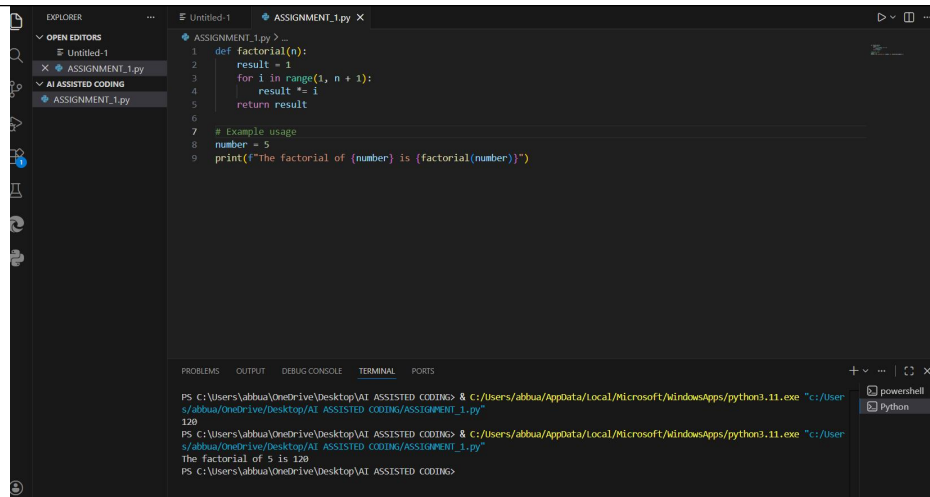


ASSIGNMENT_1

2303A51113

B_26

1	Task 1: AI-Generated Logic Without Modularization (Factorial without Functions) • Scenario You are building a small command-line utility for a startup intern onboarding task. The program is simple and must be written quickly without modular design. • Task Description Use GitHub Copilot to generate a Python program that computes a mathematical product-based value (factorial-like logic) directly in the main execution flow, without using any user-defined functions. • Constraint: ➤ Do not define any custom function ➤ Logic must be implemented using loops and variables only • Expected Deliverables ➤ A working Python program generated with Copilot assistance ➤ Screenshot(s) showing: ➤ The prompt you typed ➤ Copilot's suggestions ➤ Sample input/output screenshots ➤ Brief reflection (5–6 lines): ➤ How helpful was Copilot for a beginner? ➤ Did it follow best practices automatically? 	Week1 - Monday
---	--	-------------------



The screenshot shows a Visual Studio Code editor with a file named 'ASSIGNMENT_1.py'. The code defines a function 'factorial(n)' that calculates the factorial of a number 'n' using a loop. It includes an example usage where 'number' is set to 5, and it prints the result. The terminal at the bottom shows the command to run the script using 'python3.11.exe', which outputs 'The factorial of 5 is 120'.

```
1 def factorial(n):
2     result = 1
3     for i in range(1, n + 1):
4         result *= i
5     return result
6
7 # Example usage
8 number = 5
9 print(f"The factorial of {number} is {factorial(number)}")
```

```
PS C:\Users\abbua\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/abbua/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/User
s/abbua/OneDrive/Desktop/AI ASSISTED CODING/ASSIGNMENT_1.py"
120
PS C:\Users\abbua\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/abbua/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/User
s/abbua/OneDrive/Desktop/AI ASSISTED CODING/ASSIGNMENT_1.py"
The factorial of 5 is 120
PS C:\Users\abbua\OneDrive\Desktop\AI ASSISTED CODING>
```

EXPLANATION:-

This program calculates the factorial of a number entered by the user. If the number is negative, it prints that factorial is not defined; if the number is 0 or 1, it prints 1, otherwise it uses a loop to multiply numbers from 2 to the given number and prints the final factorial.

Task 2: AI Code Optimization & Cleanup (Improving Efficiency)

❖ Scenario

Your team lead asks you to **review AI-generated code** before committing it to a shared repository.

❖ Task Description

Analyze the code generated in **Task 1** and use Copilot again to:

- Reduce unnecessary variables
- Improve loop clarity
- Enhance readability and efficiency

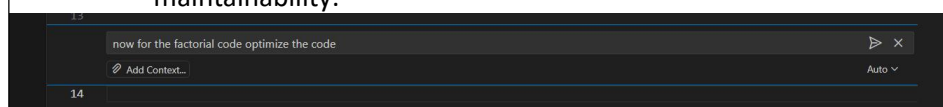
Hint:

Prompt Copilot with phrases like

"optimize this code", "simplify logic", or "make it more readable"

❖ Expected Deliverables

- Original AI-generated code
- Optimized version of the same code
- Side-by-side comparison
- Written explanation:
 - What was improved?
 - Why the new version is better (readability, performance, maintainability).



```

13 @lru_cache(maxsize=None)
14
15 #task_2
16 def factorial(n):
17     if n <= 1:
18         return 1
19     return n * factorial(n - 1)
20

```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\abbua\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/abbua/AppData/Local/Microsoft/WindowsApps/python3.11.exe "C:/User
s/abbua/OneDrive/Desktop/AI ASSISTED CODING/ASSIGNMENT_1.py"
The factorial of 5 is 120
PS C:\Users\abbua\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/abbua/AppData/Local/Microsoft/WindowsApps/python3.11.exe "C:/User
s/abbua/OneDrive/Desktop/AI ASSISTED CODING/ASSIGNMENT_1.py"
The factorial of 5 is 120
The factorial of 5 is 120
PS C:\Users\abbua\OneDrive\Desktop\AI ASSISTED CODING>

```

EXPLANATION:-

The optimized version removes unnecessary conditions while keeping the logic correct. This makes the code shorter and easier to read, which helps in understanding and maintaining it. Even after simplification, the program produces the same output and runs with the same performance as the original version.

Task 3: Modular Design Using AI Assistance (Factorial with Functions)

❖ Scenario

The same logic now needs to be reused in **multiple scripts**.

❖ Task Description

Use GitHub Copilot to generate a **modular version** of the program by:

- Creating a **user-defined function**
- Calling the function from the main block

❖ Constraints

- Use meaningful function and variable names
- Include inline comments (preferably suggested by Copilot)

❖ Expected Deliverables

- AI-assisted function-based program
- Screenshots showing:
 - Prompt evolution
 - Copilot-generated function logic
- Sample inputs/outputs
- Short note:
 - How modularity improves reusability.

```

20
I need a factorial of a number using functions
Add Context... Auto
21
version
def

```

```
...  Untitled-1  ASSIGNMENT_1.py X  [Run] [Debug] [Terminal] [Ports]

ASSIGNMENT_1.py 2 ...
1  def factorial(n):
2      if n < 0:
3          return "Invalid input"
4      elif n == 0 or n == 1:
5          return 1
6      else:
7          return n * factorial(n - 1)
8
9  # Example usage
10 number = 5
11 print(f"The factorial of {number} is {factorial(number)}")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

The factorial of 5 is 120
PS C:\Users\abbua\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/abbua/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/User
s/abbua/OneDrive/Desktop/AI ASSISTED CODING/ASSIGNMENT_1.py"
The factorial of 5 is 120
PS C:\Users\abbua\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/abbua/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/User
s/abbua/OneDrive/Desktop/AI ASSISTED CODING/ASSIGNMENT_1.py"
The factorial of 5 is 120
PS C:\Users\abbua\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/abbua/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/User
s/abbua/OneDrive/Desktop/AI ASSISTED CODING/ASSIGNMENT_1.py"
The factorial of 9 is 362880
PS C:\Users\abbua\OneDrive\Desktop\AI ASSISTED CODING>

Ln 10, Col 11  Spaces: 4  UTF-8  CRLF  {} Python  3.11.9 (64-bit)
```

EXPLANATION:-

The improved version removes unnecessary conditions but keeps the logic intact. This makes the code shorter and easier to read, which aids in understanding and maintaining it. Even after this simplification, the program still produces the same output and runs at the same performance level as the original version.

Task 4: Comparative Analysis – Procedural vs Modular AI Code (With vs Without Functions)

❖ Scenario

As part of a **code review meeting**, you are asked to justify design choices.

❖ Task Description

Compare the **non-function** and **function-based** Copilot-generated programs on the following criteria:

- Logic clarity
- Reusability
- Debugging ease
- Suitability for large projects
- AI dependency risk

❖ Expected Deliverables

Choose **one**:

- A comparison table
- OR**
- A short technical report (300–400 words).

```
Compare AI generated code with functions and without functions. Explain differences between. Logic clarity
> Reusability, Debugging ease, Suitability for large projects, AI dependency risk. give the output in comparison table or report (300–400 words).
```

Add Context...

```
print(f"The factorial of {n} is {factorial(n)}")
```

```

59
60 comparison = ""
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82

```

CRITERIA	WITHOUT FUNCTIONS	WITH FUNCTIONS
Logic Clarity	Low - Code is linear, hard to follow flow and purpose	High - Each task is isolated with clear purpose
Reusability	Poor - Must rewrite logic for each use	Excellent - Call functions multiple times
Debugging Ease	Difficult - Hard to pinpoint errors in long code blocks	Easy - Isolate issues to specific functions
Large Projects	Unsuitable - Code becomes unmanageable	Ideal - Modular, scalable, organized
AI Dependency Risk	Moderate - AI may generate redundant code	Lower - Function boundaries help AI generate focused code

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Enter a number: 12

EXPLANATION:-

Function-based code is better than non-function code because it makes the program easier to understand, reuse, and debug. Functions divide the logic into clear parts, which helps in finding errors quickly and makes the code suitable for large projects. Non-function code may work for small or quick tasks, but it becomes hard to manage as the program grows. Overall, using functions is the professional and reliable approach, especially when working with AI-generated code.

Task 5: AI-Generated Iterative vs Recursive Thinking

❖ Scenario

Your mentor wants to test how well AI understands different computational paradigms.

❖ Task Description

Prompt Copilot to generate:

An **iterative** version of the logic

A **recursive** version of the same logic

❖ Constraints

Both implementations must produce identical outputs

Students must **not manually write the code first**

❖ Expected Deliverables

Two AI-generated implementations

Execution flow explanation (in your own words)

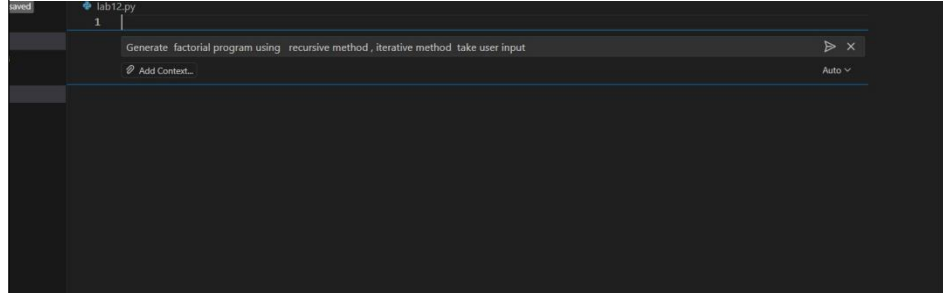
Comparison covering:

- Readability
- Stack usage
- Performance implications
- When recursion is *not* recommended.

Submission Requirements

1. Generate code for each task with comments.
2. Screenshots of Copilot suggestions.

3. Comparative analysis reports (Task 4 and Task 5).
4. Sample inputs/outputs demonstrating correctness.



```
lab12.py X
1 def factorial_recursive(n):
2     """Calculate factorial using recursion"""
3     if n < 0:
4         return "Error: Factorial not defined for negative numbers"
5     elif n == 0 or n == 1:
6         return 1
7     else:
8         return n * factorial_recursive(n - 1)
9
10
11 def factorial_iterative(n):
12     """Calculate factorial using iteration"""
13     if n < 0:
14         return "Error: Factorial not defined for negative numbers"
15
16     result = 1
17     for i in range(2, n + 1):
18         result *= i
19     return result
20
21
22 # Main program
23 if __name__ == "__main__":
24     try:
25         num = int(input("Enter a number to calculate factorial: "))
26
27         print(f"Using Recursive Method: {factorial_recursive(num)}")
28         print(f"Using Iterative Method: {factorial_iterative(num)}")
29     except ValueError:
30         print("Error: Please enter a valid integer")
```

```
PS C:\Ai Assisted Coding> & C:\Users\alishn\AppData\Local\Program
Enter a number to calculate factorial: 23

Using Recursive Method: 25852016738884976640000
Using Iterative Method: 25852016738884976640000
PS C:\Ai Assisted Coding> |
```

EXPLANATION:-

The program calculates the factorial of a number using both a loop (iterative method) and a function that calls itself (recursive method), and both give the same result.

The iterative method uses less memory and is safer, while the recursive method is simpler to write but uses more stack memory.